

Review Report

Project: OSMAN

Reviewers: Succession

Section

What You Should Write

1. Methodology Under Review	İncelediğiniz tasarım yöntemi (ust seviye mimari & hangi senaryolar var)
2. Technological Context	Bu senaryo ile ilgili gereksinimler etc. Seçtiğiniz tasarım örüntüleri
3. Summary of the Method	Yöntemi kısaca anlatın (en fazla 5–6 cümle)
4. System Evaluation	Bu yöntem bir “sistem” olarak ne kadar iyi tanımlanmış? (roller, süreçler, mimari tasarım ve tasarım örüntüleri ile ilgili akış net mi?)
5. Assumption Validity	Bu yöntemin hangi varsayımları artık geçerli değil? Hangileri hâlâ geçerli?
6. Identified Weaknesses	Tasarım yöntemi nerede zorlanıyor?
7. Missing Mechanisms	Bu durumda eksik olan ne var?
8. Scenario Evaluation	Gerçekçi bir örnek verin ve yöntemin bu durumda nasıl davrandığını açıklayın
9. Improvement Assessment	Önerilen çözüm mantıklı mı? İşe yarar mı? Neden?
10. Implementation Assessment	Yapılan küçük uygulama/prototip fikri destekliyor mu?
11. Overall Strengths	Yöntemin bu bağlamda güçlü yönleri
12. Overall Limitations	En önemli zayıf yönleri Gözlemlerinizi doğrultusunda tavsiyeler
13. Recommendation	Tasarımı kabul eder misiniz: ☐ Accept ☐ Weak Accept ☐ Borderline ☐ Weak Reject ☐ Reject
14. Justification	Neden bu kararı verdiniz? (3–5 cümle)

1. Methodology Under Review

OSMAN, **monolitik mimari** ile geliştirilmiş Java tabanlı bir statik site oluşturucudur. Kullanıcı config.toml ile ayarları yapar, Content/ klasörüne içeriklerini ekler ve Builder.java'yı çalıştırarak Output/ klasöründe statik HTML sayfaları üretir. Sistemde **Factory** ve **Strategy** tasarım örüntüleri kullanılmıştır.

Senaryolar

1. **Tema Seçimi ve Konfigürasyon:** Kullanıcı config.toml dosyasını düzenleyerek tema, başlık, alt başlık, açıklama gibi özellikleri ayarlar.
2. **İçerik Yükleme ve Site Oluşturma:** Kullanıcı metin/resim dosyalarını Content/ klasörüne ekler ve Builder.java'yı çalıştırır.
3. **Site Modifikasyonu:** Kullanıcı oluşturulan siteyi her an modifiye edebilir.

4. Hızlı Kurulum/Kaldırma: Sistem bağımlıksız, JVM üzerinden çalışır.

2. Technological Context

Gereksinimler

Projenin temel gereksinimleri şunlardır:

- Kullanıcılar config.toml veya geliştirilecek bir arayüz ile tema ve site özelliklerini ayarlayabilmeli
- Metin ve resim dosyaları pratik şekilde sisteme yüklenebilmeli
- Sistem saniyeler içinde (≤ 1.7 sn) çalışıp sonuç verebilmeli
- İşletim sisteminden bağımsız JVM üzerinden çalışabilmeli
- Sınırsız sayıda metin/resim dosyası desteklemeli

Teknolojiler

- **Backend:** Java (JVM platform bağımsızlığı)
- **Frontend:** HTML, CSS, JavaScript
- **Konfigürasyon:** TOML formatı
- **Tasarım Örüntüleri:** Factory Pattern (strateji seçimi), Strategy Pattern (config işleme mantığının kapsüllenmesi)
— NavBarStrategy, SocialLinksStrategy, ThemeNameStrategy, NonArrayStrategy)

3. Summary of the Method

OSMAN, Java ile geliştirilmiş monolitik yapıda bir statik site oluşturunucudur.

Kullanıcı config.toml dosyasında tema, başlık, sosyal medya bağlantıları gibi ayarları belirler, ardından metin dosyalarını Content/ klasörüne yükler. Builder.java çalıştırıldığında config dosyası okunur, Templates/ klasöründeki HTML/CSS şablonları Factory ve Strategy tasarım örüntüleri kullanılarak config değerleriyle doldurulur ve son çıktı Output/ klasörüne yazılır. Sistem ek kütüphane gerektirmeksizin, JVM yüklü herhangi bir platformda saniyeler içinde çalışarak statik HTML sayfaları üretmeyi hedefler.

4. System Evaluation

Akış config.toml → Builder.java → Output/ olarak net tanımlıdır. Factory ve Strategy örüntüleri doğru soyutlanmıştır. Ancak buildSite() ve makeFile() fonksiyonları tamamlanmamış olup exception fırlatarak durmaktadır. 4 stratejiden 2'si (NavbarStrategy, ThemeNameStrategy) boş bırakılmıştır. Tüm sınıfların tek dosyada olması SRP'yi ihlal eder.

5. Assumption Validity

Hâlâ geçerli:

- Statik siteler veritabanı gerektirmez
- JVM platform bağımsızlığı sağlar
- TOML konfigürasyon için uygundur

Artık zayıf:

Varsayım	Değerlendirme
Sade TOML editörü yeterlidir	Modern kullanıcılar GUI veya web arayüzü bekler
Monolitik tek dosya yeterlidir	Ölçeklenebilirlik ve bakım açısından sorunlu; tek Builder.java dosyasında tüm sınıflar var
Markdown desteği yerel olarak sağlanabilir	Proje Markdown'dan HTML'e dönüşüm için herhangi bir mekanizma içermiyor
1.7 saniyelik hedef güvenilir ölçülebilir	Benchmark veya test verisi yok

6. Identified Weaknesses

- buildSite() ve makeFile() **tamamlanmamış** — throw new Exception("HAZIR DEĞİL") ile sonlanıyor
- **TOML parser yok** — indexOf/substring ile ilkel string ayrıştırma yapıyor
- Tüm sınıflar **tek dosyada** barınıyor
- **Unit test yok**
- Hata mesajları Türkçe ve İngilizce karışık yazılmış; hata sınıflandırması yapılmamış; tüm hatalar genel Exception olarak fırlatılmaktadır

7. Missing Mechanisms

- Markdown → HTML dönüştürücü
 - Standart TOML parser kütüphanesi
 - Unit test altyapısı
 - CLI / GUI arayüzü
 - Build aracı (Maven/Gradle)
 - Resim işleme mekanizması
-

8. Scenario Evaluation

Senaryo: Bir blog yazarı OSMAN ile site kurmak istiyor.

Kullanıcı repoyu indirir, config.toml'u düzenler, yazılarını Content/ klasörüne ekler ve Builder.java'yı çalıştırır. Ancak buildSite() fonksiyonu tamamlanmamış olduğundan sistem **exception fırlatarak durur** ve hiçbir çıktı üretilmez. Kullanıcı site oluşturamaz.

9. Improvement Assessment

Factory + Strategy seçimi bu bağlam için **doğru ve mantıklıdır**. Ancak switch-case tabanlı Factory Open-Closed Principle'i ihlal eder. Monolitik yapı basitliği sağlar ama sınıfların ayrı dosyalara bölünmesi gerekir. TOML işleme için standart bir kütüphane (toml4j) kullanılması güvenilirliği artıracaktır.

10. Implementation Assessment

Prototip fikri **kısmen destekler**. readFile, writeFile, parseContentFiles gibi yardımcı metotlar çalışır durumda. SocialLinksStrategy ve NonArrayStrategy template doldurma mantığının uygulanabilirliğini gösterir. Ancak ana fonksiyonlar (buildSite, makeFile) tamamlanmamış olduğundan uçtan uca bir akış gösterilemez — prototip MVP seviyesine ulaşamamıştır.

11. Overall Strengths

- İyi tanımlanmış ve odaklı proje kapsamı
- Factory + Strategy örüntülerinin doğru seçimi
- Kapsamlı dokümantasyon (design document, requirements, UML, sprint belgeleri)
- Platform bağımsızlığı (JVM)
- Okunabilir TOML konfigürasyonu
- osman.guru tanıtım sitesi

12. Overall Limitations

- Temel fonksiyonlar tamamlanmamış (buildSite, makeFile)
- TOML parser eksikliği
- Tüm sınıflar tek dosyada
- Unit test yok
- Markdown → HTML dönüşümü eksik
- README yeterli bilgi içermiyor
- GUI/Web arayüzü eksik
- Hata mesajları Türkçe-İngilizce karışık

13. Recommendation

1. buildSite() ve makeFile() fonksiyonlarını tamamlayın
2. Boş kalan stratejileri implement edin
3. Standart TOML parser entegre edin
4. Sınıfları ayrı dosyalara bölün ve Maven/Gradle ekleyin
5. Unit testler yazın

Tasarımı kabul eder misiniz:

Weak Accept

14. Justification

OSMAN projesi kavramsal düzeyde iyi düşünülmüş bir mimari tasarıma sahiptir; Factory ve Strategy örüntülerinin seçimi doğrudur ve dokümantasyon kalitesi yüksektir. Ancak projenin temel fonksiyonları (buildSite, makeFile) tamamlanmamış olup çalıştırıldığında exception fırlatarak durur. For döngüsündeki mantık hatası ve iki stratejinin boş bırakılması, prototipin çalışır bir MVP olmadığını göstermektedir. Bu nedenle projeyi mimari tasarım başarısını takdir ederek ancak uygulama eksikliklerinin tamamlanması halinde weak accept olarak değerlendiriyoruz.